

FAST — NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES

Department of Artificial Intelligence & Data Science · Karachi Campus

PROGRAMMING FOR ARTIFICIAL INTELLIGENCE

Laboratory Manual

LAB 05

Object-Oriented Programming

Classes · Objects · Constructors · Attributes and Methods
Encapsulation · Inheritance · Method Overriding · Polymorphism

PREPARED BY

Khadeejah Ashraf

Lab Instructor | Department of Artificial Intelligence & Data Science

Contents

1. Lab Objectives
2. Lab Rules and Conduct
3. From Functions to Objects
4. Classes and Objects
5. The Constructor and Instance Attributes
6. Methods and `self`
7. Printing an Object: `__str__` and Friends
8. Encapsulation
9. Inheritance
10. Method Overriding and `super()`
11. Polymorphism
12. Abstract Base Classes
13. Designing Classes Well, and Common Errors
14. Quick Reference
15. Class Tasks (to be completed during the session)
16. Home Tasks
17. Submission and Assessment

1. Lab Objectives

By the end of this session, you should be able to:

- Explain what an **object** is, and why bundling data with the code that works on it is different from passing dictionaries into functions.
- Define a class, create objects from it, and distinguish a **class attribute** from an **instance attribute**.
- Write a constructor `__init__`, including default parameter values, and explain what `self` refers to.
- Add methods to a class and call them on an object, and explain why `self` is written in the definition but not in the call.
- Give a class a readable printed form with `__str__`, and compare objects with `__eq__`.
- Apply encapsulation: `public`, `_protected` and `__private` members, and expose controlled access with getters, setters or `@property`.
- Derive one class from another, and describe how Python searches for an attribute along the inheritance chain.
- Override a method in a child class and extend the parent's version with `super()`.
- Recognise single, multilevel, hierarchical and multiple inheritance, and read a simple method resolution order.
- Explain polymorphism and duck typing, and write one function that works on objects of several unrelated classes.
- Declare an abstract base class with `ABC` and `@abstractmethod`, and say why it cannot be instantiated.

2. Lab Rules and Conduct

- Maintain discipline throughout the lab session.
- Listen carefully and follow the instructions as they are given.
- Raise your hand if you face any problem; do not leave your seat.
- **Class tasks** must be completed and demonstrated during the session itself.
- **Home tasks** must be submitted before the deadline announced on the course portal.
- Get your work checked by the instructor before the end of the session.

Reminder: work that is not checked during the session carries no marks, even if the code runs correctly.

3. From Functions to Objects

In Lab 04 you kept a student's data in a dictionary and wrote separate functions that took that dictionary as an argument. It works, but the arrangement has a weakness: nothing ties the data to the functions that understand it. Any part of the program can misspell a key, add a field that the functions do not expect, or call `average()` on a dictionary that has no marks at all.

The procedural version.

```
student = {'name': 'Ali', 'marks': [78, 45, 90]}

def average(record):
    return round(sum(record['marks']) / len(record['marks']), 2)

print(average(student))
student['mrks'] = []      # a typo nothing will catch until much later
```

The object-oriented version. The data and the operations live in one unit, and every object is guaranteed to be built the same way.

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def average(self):
        return round(sum(self.marks) / len(self.marks), 2)

ali = Student('Ali', [78, 45, 90])
print(ali.name, ali.average())
```

Output

```
Ali 71.0
```

Object-oriented programming gives you	Which means in practice
Data and behaviour together	<code>ali.average()</code> needs no second argument — the object already knows its own marks.
A guaranteed shape	Every <code>Student</code> is built by the same constructor, so every one of them has name and marks.
A vocabulary	The program starts to talk about students, accounts and vehicles instead of dictionaries and lists.
Reuse through inheritance	A <code>PostgraduateStudent</code> can keep everything a <code>Student</code> does and change only what differs.
Substitutability	Code that works on a <code>Student</code> keeps working on any class that behaves like one.

You have used objects all along: a list is an object of class `list`, and `marks.append(5)` is a method call on it. This lab is about writing your own classes.

4. Classes and Objects

A **class** is the blueprint; an **object** (or **instance**) is one thing built from that blueprint. The class is written once; objects are created from it as often as needed, and each one carries its own data.

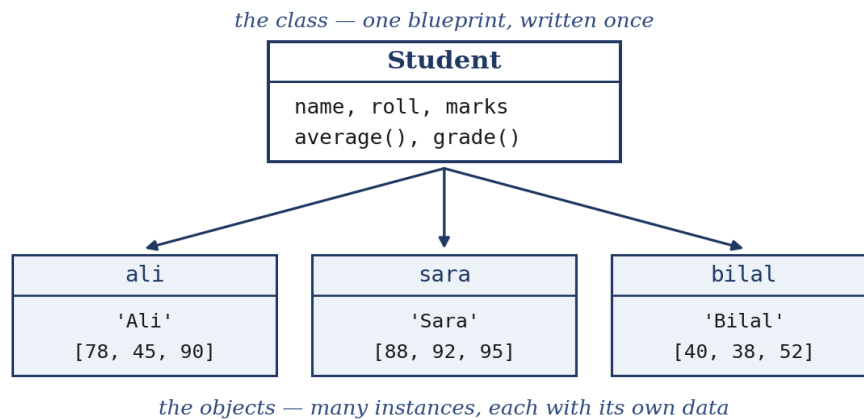


Figure 4.1 One class, three objects — each instance keeps its own attribute values

4.1 The Anatomy of a Class

```

class Student: _____ class header — a capitalised noun

    school = 'FAST NUCES' _____ class attribute — shared by every object

    def __init__(self, name, marks): _____ the constructor, run on every new object

        self.name = name _____ instance attribute — this object's own data

        self.marks = marks

    def average(self): _____ a method; self is the object it was called on

        return sum(self.marks) / len(self.marks) - methods read the object through self
  
```

Figure 4.2 The parts of a class definition

By convention a class name is a singular noun in CapWords: `Student`, `BankAccount`, `RentalVehicle`. The body is indented, exactly like the body of a function.

4.2 Creating Objects

Calling the class as though it were a function creates an object. Each call produces a separate object with its own attributes, and `type()` reports which class it came from.

```

class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

ali = Student('Ali', [78, 45, 90])
sara = Student('Sara', [88, 92, 95])

print(ali.name, sara.name)
print(type(ali))
print(isinstance(ali, Student))
print(ali is sara)          # two different objects
  
```

Output

```
Ali Sara
<class '__main__.Student'>
True
False
```

4.3 Class Attributes and Instance Attributes

An attribute assigned in the class body belongs to the **class**, and every object sees the same one. An attribute assigned through `self` belongs to that **instance** alone. Class attributes are for facts that are true of every object — a shared constant, a counter of how many objects exist.

```
class Student:
    school = 'FAST NUCES'      # class attribute - shared
    count = 0                 # shared counter

    def __init__(self, name):
        self.name = name      # instance attribute - one per object
        Student.count += 1    # update the shared value

ali = Student('Ali')
sara = Student('Sara')

print(ali.school, sara.school) # the same string for both
print(Student.count)          # two objects have been created
print(ali.name, sara.name)    # but each has its own name
```

Output

```
FAST NUCES FAST NUCES
2
Ali Sara
```

A trap: `ali.school = 'Other'` does not change the class attribute — it creates a **new instance attribute** on `ali` that hides it. Assign through the class name, `Student.school = ...`, when you mean to change the shared value.

5. The Constructor and Instance Attributes

`__init__` is the **constructor**: Python calls it automatically on every new object, and its job is to set that object's starting attributes. It is never called directly, and it must not return anything.

```
class BankAccount:
    def __init__(self, holder, balance=0):
        self.holder = holder
        self.balance = balance
        self.history = []      # every account starts with its own empty list

a = BankAccount('Ali', 5000)
b = BankAccount('Sara')      # balance takes its default

print(a.holder, a.balance)
print(b.holder, b.balance)
print(a.history is b.history) # separate lists, not a shared one
```

Output

```
Ali 5000
Sara 0
False
```

Why build the list inside: writing `history = []` in the class body would make one list shared by every account — the same trap as the mutable default argument in Lab 04. Build per-object data inside `__init__`.

5.1 Validating in the Constructor

Because every object passes through `__init__`, it is the natural place to reject impossible data — a negative balance, an empty name — so that no badly formed object ever exists.

```
class BankAccount:
    def __init__(self, holder, balance=0):
        if balance < 0:
            raise ValueError("balance cannot be negative")
        self.holder = holder
        self.balance = balance

try:
    BankAccount('Ali', -100)
except ValueError as err:
    print("rejected:", err)
```

Output

```
rejected: balance cannot be negative
```

6. Methods and self

A **method** is a function defined inside a class. Its first parameter is always `self`, which is the object the method was called on. You write `self` in the definition but never pass it at the call — Python supplies it automatically, so `ali.average()` runs `Student.average(ali)`.

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def average(self):
        # reads the object's own data
        return round(sum(self.marks) / len(self.marks), 2)

    def add_mark(self, mark):
        # changes the object's own data
        self.marks.append(mark)

    def grade(self):
        # a method may call another method
        avg = self.average()
        if avg >= 80:
            return 'A'
        elif avg >= 70:
            return 'B'
        elif avg >= 60:
            return 'C'
        return 'F'

ali = Student('Ali', [78, 45, 90])
print(ali.average(), ali.grade())

ali.add_mark(100)
print(ali.marks, ali.average(), ali.grade())

print(Student.average(ali))          # the same call, written the long way
```

Output

```
71.0 B
[78, 45, 90, 100] 78.25 B
78.25
```

Kind of method	How it is written	Typical use
Instance method	First parameter <code>self</code>	Almost everything — reads or changes one object
Class method	<code>@classmethod</code> , first parameter <code>cls</code>	An alternative constructor, or work on a class attribute
Static method	<code>@staticmethod</code> , no <code>self</code>	A helper that logically belongs to the class but needs no object

```
class Student:
    count = 0

    def __init__(self, name):
        self.name = name
        Student.count += 1

    @classmethod
    def how_many(cls):
        return cls.count

    @staticmethod
    def is_passing(mark):
        return mark >= 50

Student('Ali'); Student('Sara')
print(Student.how_many())
print(Student.is_passing(45), Student.is_passing(78))
```

Output

```
2
False True
```

Forgetting `self`: `def average():` inside a class gives `TypeError: average() takes 0 positional arguments but 1 was given` when you call it on an object — Python passed the object and the method had nowhere to put it.

7. Printing an Object: `__str__` and Friends

Printing an object straight out of the box gives an unhelpful line such as `<__main__.Student object at 0x7f...>`. Methods whose names begin and end with two underscores — **dunder** methods — let your class hook into the language's own syntax. `__str__` decides what `print()` shows.

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def __str__(self):
        return "Student(" + self.name + ", " + str(len(self.marks)) + " marks)"

    def __len__(self):
```

```

        return len(self.marks)

    def __eq__(self, other):
        return self.name == other.name

ali = Student('Ali', [78, 45, 90])
clone = Student('Ali', [10, 20])

print(ali)           # uses __str__
print(len(ali))       # uses __len__
print(ali == clone)   # uses __eq__

```

Output

```

Student(Ali, 3 marks)
3
True

```

Dunder method	Triggered by	Should return
<code>__init__(self, ...)</code>	<code>Student(...)</code>	Nothing — it only sets attributes
<code>__str__(self)</code>	<code>print(obj), str(obj)</code>	A readable string for a human
<code>__repr__(self)</code>	The object shown in a notebook cell	An unambiguous string for a programmer
<code>__len__(self)</code>	<code>len(obj)</code>	An integer
<code>__eq__(self, other)</code>	<code>obj1 == obj2</code>	True or False
<code>__add__(self, other)</code>	<code>obj1 + obj2</code>	A new object

8. Encapsulation

Encapsulation means keeping an object's data and the code that maintains it in one place, and controlling what the outside world may touch directly. Python marks intent by naming rather than by keywords.

Written as	Meaning	Enforced?
<code>self.balance</code>	Public — anyone may read and change it	—
<code>self._balance</code>	Protected by convention — internal, but subclasses may use it	No, it is a signal to the reader
<code>self.__balance</code>	Private — intended for this class only	Partly: the name is mangled to <code>_ClassName__balance</code>

```

class BankAccount:
    def __init__(self, holder, balance=0):
        self.holder = holder          # public
        self._bank = 'HBL'           # protected by convention
        self.__balance = balance      # private

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("deposit must be positive")
        self.__balance += amount

```

```

    def get_balance(self):          # a getter: controlled read access
        return self.__balance

a = BankAccount('Ali', 5000)
a.deposit(1500)
print(a.holder, a.get_balance())
print(a._bank)                    # possible, but you are trespassing
print(a.__balance)                # AttributeError

```

Output

```

Ali 6500
HBL
AttributeError: 'BankAccount' object has no attribute '__balance'

```

Name mangling, not security: the attribute is still reachable as `a._BankAccount__balance`. Python protects against **accidents**, not against a determined programmer. The double underscore says “do not rely on this from outside”.

8.1 Getters, Setters and @property

A getter and a setter are ordinary methods that guard an attribute. The `@property` decorator lets them be used with the plain attribute syntax, so the class can add validation later without every caller having to change.

```

class Student:
    def __init__(self, name, marks=0):
        self.name = name
        self.__marks = marks

    @property
    def marks(self):          # reading obj.marks runs this
        return self.__marks

    @marks.setter
    def marks(self, value):   # assigning obj.marks = x runs this
        if not 0 <= value <= 100:
            raise ValueError("marks must be between 0 and 100")
        self.__marks = value

s = Student('Ali')
s.marks = 78                  # looks like an attribute, runs the setter
print(s.marks)

try:
    s.marks = 150
except ValueError as err:
    print("rejected:", err)

```

Output

```

78
rejected: marks must be between 0 and 100

```

9. Inheritance

Inheritance lets a new class start from an existing one. The new class — the **child**, **derived** or **subclass** — automatically has every attribute and method of the **parent**, **base** or **superclass**, and adds or replaces only what is different. The relationship to look for is “is a”: a **Bus is a Vehicle**, so **Bus** should inherit from **Vehicle**.

```

class Vehicle:
    def __init__(self, make, seats):

```

```

        self.make = make
        self.seats = seats

    def fare(self):
        return self.seats * 100

    def describe(self):
        return self.make + " with " + str(self.seats) + " seats"

class Bus(Vehicle):
    pass

b = Bus('Hino', 50)
print(b.describe())
print(b.fare())
print(isinstance(b, Bus), isinstance(b, Vehicle), issubclass(Bus, Vehicle))

```

Output

```

Hino with 50 seats
5000
True True True

```

9.1 How Python Finds an Attribute

When you write `obj.something`, Python looks first at the object itself, then at its class, then at that class's parent, and so on up the chain. The first match wins, and if nothing matches you get an `AttributeError`.

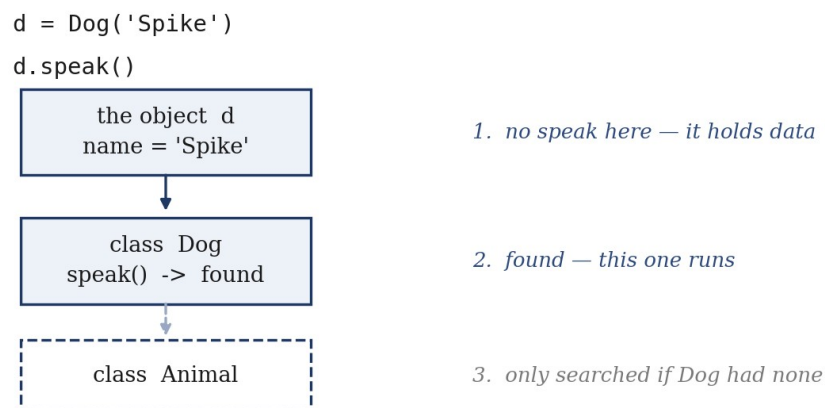


Figure 9.1 The search for a method starts at the object and moves up the chain

9.2 Extending the Parent

A child class usually adds attributes of its own. Its constructor takes the extra values, hands the shared ones up to the parent with `super().__init__(...)`, and then sets what is new.

```

class Bus(Vehicle):
    def __init__(self, make, seats, route):
        super().__init__(make, seats)    # let Vehicle set make and seats
        self.route = route               # then add what is new to Bus

    def describe(self):
        return super().describe() + " on route " + self.route

b = Bus('Hino', 50, 'Karachi-Hyderabad')

```

```
print(b.make, b.seats, b.route)
print(b.describe())
```

Output

```
Hino 50 Karachi-Hyderabad
Hino with 50 seats on route Karachi-Hyderabad
```

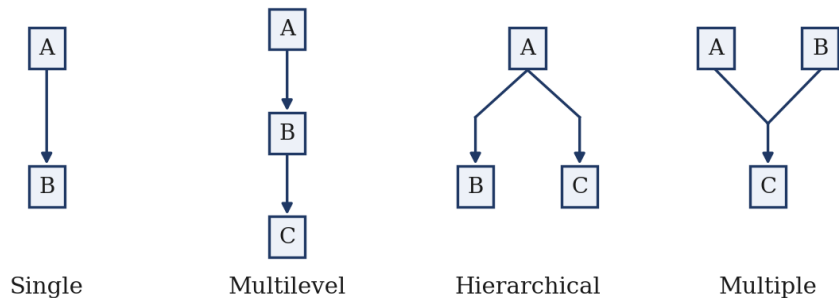
9.3 The Four Shapes of Inheritance

Figure 9.2 Single, multilevel, hierarchical and multiple inheritance

Shape	Meaning	Example
Single	One child, one parent	<code>class Bus(Vehicle)</code>
Multilevel	A chain: a child of a child	<code>Vehicle → Car → SportsCar</code>
Hierarchical	Several children of one parent	<code>Car, Bus, Truck from Vehicle</code>
Multiple	One child, several parents	<code>class Amphibian(Car, Boat)</code>

```
# multilevel: each class builds on the one above it
class Vehicle:
    def __init__(self, make):
        self.make = make

class Car(Vehicle):
    def __init__(self, make, doors):
        super().__init__(make)
        self.doors = doors

class SportsCar(Car):
    def __init__(self, make, doors, top_speed):
        super().__init__(make, doors)
        self.top_speed = top_speed

s = SportsCar('Toyota', 2, 250)
print(s.make, s.doors, s.top_speed)
print(SportsCar.__mro__)
```

Output

```
Toyota 2 250
(<class '__main__.SportsCar'>, <class '__main__.Car'>, <class '__main__.Vehicle'>,
<class 'object'>)
```

`__mro__` is the **method resolution order**: the exact sequence Python searches. With multiple inheritance it is the rule that decides which parent wins when both define the same method — the parents are searched left to right, in the order they were written in the class header.

```

class Engine:
    def start(self):
        return "engine started"

class Radio:
    def start(self):
        return "radio started"

class Car(Engine, Radio):          # Engine is searched first
    pass

print(Car().start())
print([c.__name__ for c in Car.__mro__])

```

Output

```

engine started
['Car', 'Engine', 'Radio', 'object']

```

Use multiple inheritance sparingly: two parents that define the same method make a class hard to reason about. Most real designs use single or multilevel inheritance.

10. Method Overriding and super()

Overriding means a child class defines a method that its parent already has. Because the search starts at the child, the child's version is the one that runs. This is how a subclass changes behaviour without touching the parent's code.

```

class Vehicle:
    def __init__(self, make, seats):
        self.make, self.seats = make, seats

    def fare(self):
        return self.seats * 100

class Bus(Vehicle):
    def fare(self):                # overrides Vehicle.fare
        return self.seats * 100 * 1.10  # 10% maintenance charge

class Rickshaw(Vehicle):
    def fare(self):                # a different override
        return self.seats * 60

for v in [Vehicle('Generic', 4), Bus('Hino', 50), Rickshaw('Sazgar', 3)]:
    print(type(v).__name__, v.fare())

```

Output

```

Vehicle 400
Bus 5500.0
Rickshaw 180

```

10.1 Extending Rather Than Replacing

An override does not have to throw the parent's work away. `super()` calls the parent's version from inside the child's, so shared logic stays in one place.

```

class Bus(Vehicle):

```

```
def fare(self):
    base = super().fare()          # reuse Vehicle's calculation
    return base + base * 0.10      # then add what is specific to a Bus

print(Bus('Hino', 50).fare())
```

Output

```
5500.0
```

Written as	Effect
Child defines a new method	Extends the parent — the object can do more
Child redefines a parent method	Overrides it — the child's version runs
Child override calls <code>super().method()</code>	Extends the parent's behaviour instead of discarding it
Child does neither	Inherits the parent's method unchanged

11. Polymorphism

Polymorphism — “many forms” — is the property that the same call means different things depending on the object it lands on. You have relied on it since Lab 01: `len()` works on a string, a list and a dictionary, and `+` adds numbers but joins strings.

```
for value in ["abc", [1, 2, 3, 4], {'a': 1}]:
    print(type(value).__name__, len(value))

print(3 + 4)
print("Prog" + "ramming")
```

Output

```
str 3
list 4
dict 1
7
Programming
```

11.1 One Loop, Many Classes

With your own classes the same effect appears the moment several classes offer a method of the same name. The loop below never asks what kind of object it is holding; each object supplies its own version.

```
class Car:
    def describe(self):
        return "A car carries a small family."

class Truck:
    def describe(self):
        return "A truck carries cargo."

class Rickshaw:
    def describe(self):
        return "A rickshaw carries three passengers."

for vehicle in [Car(), Truck(), Rickshaw()]:
    print(vehicle.describe())
```

Output

```
A car carries a small family.
A truck carries cargo.
A rickshaw carries three passengers.
```

Duck typing: notice that Car, Truck and Rickshaw share no parent. Python does not check the type — it only asks whether the object has a `describe` method. “If it walks like a duck and quacks like a duck, treat it as a duck.”

11.2 A Polymorphic Function

The same idea lets one function serve every class in a hierarchy. Adding a new subclass later needs no change to this function at all — the single most valuable property of the whole design.

```
def print_details(vehicle):
    """Work with any object that has a make and a fare()."""
    print(vehicle.make, "->", vehicle.fare())

print_details(Bus('Hino', 50))
print_details(Rickshaw('Sazgar', 3))
```

Output

```
Hino -> 5500.0
Sazgar -> 180
```

11.3 Operator Overloading

Defining a dunder such as `__add__` makes an operator work on your own objects. Used sparingly, it makes a class read like a built-in type.

```
class Money:
    def __init__(self, rupees):
        self.rupees = rupees

    def __add__(self, other):
        return Money(self.rupees + other.rupees)

    def __str__(self):
        return "Rs " + str(self.rupees)

print(Money(1500) + Money(2500))
```

Output

```
Rs 4000
```

On method overloading: Python has no compile-time overloading — two methods with the same name means the second replaces the first. Use default parameters or `*args` (Lab 04) to let one method accept different numbers of arguments.

12. Abstract Base Classes

Sometimes a parent exists only to state what its children must provide. `Shape` has no sensible `area()` of its own, but every real shape must have one. The `abc` module turns that intention into a rule the language enforces: a class deriving from `ABC` with at least one `@abstractmethod` cannot be instantiated, and a subclass that fails to implement every abstract method cannot be instantiated either.

```
from abc import ABC, abstractmethod

class Shape(ABC):
```

```

def __init__(self, name):
    self.name = name

    @abstractmethod
    def area(self):
        """Every concrete shape must supply this."""

    def describe(self):
        # a concrete method, inherited as usual
        return self.name + " has area " + str(round(self.area(), 2))

class Rectangle(Shape):
    def __init__(self, width, height):
        super().__init__('Rectangle')
        self.width, self.height = width, height

    def area(self):
        return self.width * self.height

class Circle(Shape):
    def __init__(self, radius):
        super().__init__('Circle')
        self.radius = radius

    def area(self):
        return 3.14159 * self.radius ** 2

for shape in [Rectangle(4, 5), Circle(3)]:
    print(shape.describe())

try:
    Shape('generic')
except TypeError as err:
    print("TypeError:", err)

```

Output

```

Rectangle has area 20
Circle has area 28.27
TypeError: Can't instantiate abstract class Shape without an implementation for abstract
method 'area'

```

Why it is useful: an abstract base class turns “please remember to write an `area()` method” into an error at the moment the object is created, rather than a mysterious `AttributeError` much later.

13. Designing Classes Well, and Common Errors

Design rule	In practice
A class is a noun, a method is a verb	<code>BankAccount.deposit()</code> , not <code>AccountManager.doStuff()</code>
Inherit only for “is a”	A Bus is a Vehicle. An Engine is not a Car — a car has an engine, so store it as an attribute.
Keep the constructor honest	Set every attribute the class needs in <code>__init__</code> , and reject impossible values there.

Design rule	In practice
Hide what callers should not touch	Make the balance private and change it only through <code>deposit()</code> and <code>withdraw()</code> .
Return, do not print	As in Lab 04, only the reporting method should print.
One class, one responsibility	If a class both stores a student and prints the timetable, split it in two.

Message or symptom	Usual cause
<code>TypeError: __init__() missing 1 required positional argument</code>	An argument was left out when creating the object.
<code>TypeError: f() takes 0 positional arguments but 1 was given</code>	<code>self</code> was left out of the method definition.
<code>AttributeError: 'Student' object has no attribute 'marks'</code>	The attribute is never assigned in <code>__init__</code> , or its name is misspelt.
<code>AttributeError: ... has no attribute '__balance'</code>	A private attribute accessed from outside the class; use a getter or <code>@property</code> .
<code>NameError: name 'self' is not defined</code>	<code>self</code> used outside a method, or in the class body rather than inside <code>__init__</code> .
<code><__main__.Student object at 0x...> is printed</code>	The class has no <code>__str__</code> method.
Every object shares the same list	A mutable value assigned in the class body instead of inside <code>__init__</code> .
<code>TypeError: Can't instantiate abstract class ...</code>	An abstract method was not implemented in the subclass.

14. Quick Reference

Form	Meaning	Example
<code>class Student:</code>	Define a class	CapWords, singular noun
<code>def __init__(self, ...):</code>	Constructor, run on every new object	<code>self.name = name</code>
<code>obj = Student('Ali')</code>	Create an object	Calls <code>__init__</code> for you
<code>self</code>	The object the method was called on	First parameter of every method
<code>school = 'FAST' in the body</code>	Class attribute — shared	<code>Student.school</code>
<code>self.name = name</code>	Instance attribute — per object	<code>ali.name</code>
<code>@classmethod cls</code>	A method that works on the class	<code>Student.how_many()</code>
<code>@staticmethod</code>	A helper needing no object	<code>Student.is_passing(45)</code>

Form	Meaning	Example
<code>_x / __x</code>	Protected by convention / private (mangled)	<code>self.__balance</code>
<code>@property @x.setter</code>	Attribute syntax with validation behind it	<code>s.marks = 78</code>
<code>class Bus(Vehicle):</code>	Inherit from a parent	Bus gains every <code>Vehicle</code> method
<code>super().__init__(...)</code>	Call the parent's version	Inside the child's <code>__init__</code> or override
<code>isinstance(o, C)</code> <code>issubclass(B, A)</code>	Type tests	<code>isinstance(b, Vehicle)</code>
<code>C.__mro__</code>	The order in which classes are searched	Multiple inheritance
<code>__str__ __len__</code> <code>__eq__ __add__</code>	Hook into <code>print</code> , <code>len</code> , <code>==</code> , <code>+</code>	<code>print(ali)</code>
<code>ABC @abstractmethod</code>	A parent that cannot be instantiated	<code>class Shape(ABC):</code>

15. Class Tasks

These five tasks are to be completed **during the lab session** and demonstrated to the instructor before you leave. Work in the accompanying notebook `Lab05_Object_Oriented_Programming.ipynb`, in the section titled **Class Tasks**. Every class must have a docstring, every attribute must be set in the constructor, and only a display method may call `print()`.

C1. A Book Class — classes, constructors, attributes and methods. Write a class `Book` with the class attribute `library = 'FAST NUCES Library'` and the instance attributes `title`, `author`, `pages` and `issued` (which starts as `False`). Give it: `issue()`, which marks the book as issued and returns a message, or refuses if it is already out; `return_book()`, which reverses that; `is_long()`, returning `True` when the book has more than 300 pages; and `__str__`, so that printing a book shows its title, author and current status. Create three books, issue two of them, attempt to issue one of those twice, and print all three. Finally add a class attribute `count` that records how many `Book` objects have been created, and print it through a `@classmethod` called `how_many()`.

Show in one line of code that changing `Book.library` changes it for every existing object, and explain why in a comment.

C2. A Bank Account — encapsulation. Write a class `BankAccount` whose `__init__` takes an account holder's name, a starting balance (default 0) and a four-digit security code. The balance and the security code must be **private**, and the account number must be **protected**. Provide `deposit(amount)` and `withdraw(amount)` which reject a non-positive amount and a withdrawal larger than the balance by raising `ValueError`; a `balance @property` that allows reading but not direct assignment; a `statement()` method that prints the holder, the account number and the last five transactions from a private history list; and `verify(code)` returning `True` or `False` without ever revealing the stored code.

Demonstrate that `account.__balance` raises `AttributeError` from outside the class, then show that `account._BankAccount__balance` reaches it anyway, and explain in a comment what that tells you about privacy in Python.

C3. A Vehicle Hierarchy — inheritance and overriding. Write a base class `Vehicle` with `make`, `model` and `seats`, a method `fare()` returning `seats * 100`, and a `describe()` returning a one-line summary. Derive three classes: `Bus`, which overrides `fare()` to add a 10% maintenance charge; `Car`, which adds an `ac` attribute and overrides `fare()` to add 25% when the air conditioning is on; and `Truck`, which replaces `seats` with a `load_tonnes` attribute in its constructor and charges `load_tonnes * 500`. Every override of

`describe()` must call `super().describe()` rather than repeating it. Put four vehicles in a list, loop over it once, and print each one's description and fare.

Then derive `SchoolBus` from `Bus` (multilevel) with a fixed monthly fare, print `SchoolBus.__mro__`, and state in a comment which class supplies `describe()` for a `SchoolBus` object and why.

- C4. Shapes — abstraction and polymorphism.** Using `ABC` and `@abstractmethod`, write an abstract class `Shape` with the abstract methods `area()` and `perimeter()` and a concrete method `summary()` that uses both. Implement `Rectangle`, `Square` (derived from `Rectangle`, not from `Shape`), `Triangle` (given three sides, using Heron's formula) and `Circle`. Store one of each in a list, print the summary of every one with a single loop, and then print the shapes sorted from the largest area to the smallest using `sorted()` with a key.

Show that `Shape()` raises a `TypeError`, and that a subclass which forgets to implement `perimeter()` also cannot be instantiated. Include both in `try / except` blocks so the notebook still runs to the end.

- C5. A Small Library System — the whole lab in one program.** Combine the previous ideas into one program with three classes. `LibraryItem` is the abstract base with `title`, a private `__item_id`, an `is_available` flag, an abstract `loan_period_days()` and a concrete `describe()`. `PrintedBook` and `DVD` derive from it with different loan periods and their own extra attributes. `Member` stores a name, a private contact number and a list of borrowed items, with `borrow(item)` and `give_back(item)` that update both the member and the item, refuse to borrow something already out, and a `history()` method that prints what the member has taken. Finally write one function `show(entity)` that prints the details of **any** of these objects — book, DVD or member — to demonstrate polymorphism.

Your driver code must create at least two members and four items, perform a borrow, a refused borrow and a return, and end by printing every member's history.

Assessment of class tasks: you must be able to explain any line of your code — in particular why a given attribute is private, which class an inherited method comes from, and what `super()` is doing at each call. A working program that you cannot explain earns no marks.

16. Home Tasks

Attempt all the questions below. Write each program in a **separate .py file**, give every class a docstring, test it with more than one object, and submit it according to the naming convention in Section 17.

- Q1.** Write a class `Rectangle` with `length` and `width`, methods `area()` and `perimeter()`, and a `__str__` that prints the dimensions. Create three rectangles and print each one with its area and perimeter.
- Q2.** Write a class `Student` with `roll`, `name` and a list of three marks. Add `total()`, `average()`, `grade()` and `display()`. Create four students, and print the name of the topper by comparing the averages.
- Q3.** Write a class `Circle` in which the radius is a private attribute exposed through a `@property`. The setter must reject a negative radius with a `ValueError`. Add `area()` and `circumference()`, and demonstrate both a valid and an invalid assignment.
- Q4.** Create a `Bus` class that inherits from a `Vehicle` class. The default fare of any vehicle is `seating_capacity * 100`; for a bus, add a further 10% of the total as a maintenance charge, so that the final amount is the fare plus 10%. Print the fare of a car and of a bus to show the difference.
- Q5.** Create an abstract base class `Shape` with an abstract method `area()`. Derive `Rectangle`, `Triangle` and `Square` from it, implement `area()` in each, and print the area of all three from a single loop.
- Q6.** Write a class `Account` whose member variables `account_no`, `account_bal` and `security_code` are all private. Initialise them through a public method rather than in the constructor, and print all the data through another public method. Explain in a comment why the variables cannot be printed directly.
- Q7.** A university is upgrading its system. Class `Student` has a method that displays the student's id and name. Class `Marks`, derived from `Student`, has a method that displays the marks of three courses — algorithms, data science and calculus. Class `Result`, derived from `Marks`, has a method that calculates and displays the total and the average. In the driver code create one `Result` object and display the student details, the marks of each course and the total and average.

- Q8.** Write a class `Employee` with `name` and `salary` and a method `calculate_bonus()` returning 10% of the salary. Derive `Manager`, whose bonus is 20% and which has a `hire()` method, and `Developer`, which has a `write_code()` method. Then derive `SeniorManager` from `Manager` with a bonus of 30%. Put one of each in a list and print every employee's name, role and bonus from one loop.
- Q9.** Write a class `Time` storing hours and minutes, with `__str__` printing `hh:mm` and `__add__` returning a new `Time` in which 75 minutes correctly becomes one hour and fifteen minutes. Demonstrate adding three times together.
- Q10.** Write a class `Playlist` that holds a list of song titles, with `add(song)`, `remove(song)`, `__len__` so that `len(playlist)` works, and `__str__` that numbers the songs. Raise a `ValueError` when removing a song that is not there.
- Q11.** Demonstrate multiple inheritance: write `Engine` with a `start()` method, `MusicSystem` with its own `start()` method, and `Car` inheriting from both. Print what `Car().start()` does, print `Car.__mro__`, then swap the order of the parents and explain the change in a comment.
- Q12.** Develop a small vehicle-rental system. Design a class hierarchy for `Car`, `SUV` and `Truck` with the attributes `make`, `model`, `rental price per day` and `availability`. Each class must be able to check availability, calculate the cost of a rental for a given number of days, and display its details, and the availability must change when a vehicle is rented or returned. Add a `Customer` class holding a name, a private contact number and the list of vehicles rented, with a method that displays the rental history, and a `RentalReservation` class linking a customer to a vehicle with start and end dates. Finally write one function that can display the details of any vehicle or of a reservation, to demonstrate polymorphism.

17. Submission and Assessment

Follow the naming convention below so that your work can be identified and graded correctly:

- **Class tasks:** completed in the notebook `RollNo_Lab05_ClassTasks.ipynb` and demonstrated before the end of the session. Run **Kernel ► Restart & Run All** before you call the instructor.
- **Home tasks:** one script per question, named `RollNo_Lab05_Qn.py` — for example `22K-1234_Lab05_Q7.py`.
- Place all the files for this lab in a single folder and compress it before uploading.
- Every file must begin with a comment giving your name, roll number and the question number, and every class must carry a docstring.
- Code that does not run will not be marked. Test your program before calling the instructor.

Assessment Criterion	Weight
Correctness of output for all valid inputs, including rejected input	30%
Class tasks completed and demonstrated in the session	20%
Correct class design: constructors, attributes, methods, <code>self</code>	20%
Correct use of encapsulation, inheritance and overriding	20%
Readability: naming, docstrings, <code>__str__</code> where useful	10%

End of Lab 05